

Food Delivery Order Queue

ระบบจัดคิวคำสั่งซื้ออาหาร - เปรียบเทียบ FIFO Queue และ Priority Queue

Case Story: ร้านอาหารได้รับ Order จาก Food Delivery Platform หลายรายการพร้อมกัน

1. Problem Analysis

ระบบต้องจัดลำดับการเตรียมอาหารจากหลาย Order โดยแต่ละ Order มี Order ID, Order Time, Food Type, Preparation Time และ Priority เป้าหมายคือเปรียบเทียบการจัดคิว 2 แบบ: FIFO ซึ่งให้มาก่อนทำก่อน และ Priority Queue ซึ่งให้ Express มาก่อน Normal และ Large

Order ID	Order Type	Preparation Time	Priority
O1	Normal	15	2
O2	Express	5	1=สูงสุด
O3	Normal	10	2
O4	Express	8	1=สูงสุด
O5	Large	25	3

2. Queue Design

ระบบจัดคิวคำสั่งซื้ออาหารมี 2 Algorithm สำหรับเปรียบเทียบ ได้แก่ FIFO Queue และ Priority Queue

Algorithm A: FIFO Queue

ใช้หลัก First In, First Out (FIFO) คือ Order ที่เข้ามาก่อนจะได้รับการเตรียมก่อน โดยไม่คำนึงถึงประเภทของ Order

ลำดับตัวอย่าง:

O1 → O2 → O3 → O4 → O5

การทำงานหลัก:

- Enqueue** : เพิ่ม Order เข้าท้ายคิว
- Dequeue** : นำ Order ที่อยู่หน้าคิวออกไปเตรียมอาหาร

Algorithm B: Priority Queue

ใช้ Priority ในการกำหนดลำดับการเตรียมอาหาร โดยกำหนด

Express > Normal > Large

ดังนั้น Order ที่มี Priority สูงกว่าจะได้รับการเตรียมก่อน แม้จะเข้าคิวทีหลัง

ลำดับตัวอย่าง:

O2 → O4 → O1 → O3 → O5

ในกรณีที่ Priority เท่ากัน จะให้ Order ที่เข้ามาก่อนถูกเลือกก่อน เพื่อรักษาความเป็นธรรมภายใน Priority เดียวกัน

3. Pseudocode Algorithm A

Pseudocode Algorithm A: FIFO Queue

INPUT: Order list O1..On

Create empty Queue Q

FOR each order O in arrival order

 ENQUEUE O into Q

END FOR

currentTime <- 0

WHILE Q is not empty

 order <- DEQUEUE Q

 waitingTime <- currentTime

 prepare order for Preparation Time

 currentTime <- currentTime + order.PreparationTime

 record order and waitingTime

END WHILE

OUTPUT preparation sequence and waiting times

4. Pseudocode Algorithm B

Pseudocode Algorithm B: Priority Queue

INPUT: Order list O1..On

Create empty Priority Queue P

Priority: Express=1, Normal=2, Large=3

Tie-breaker: earlier arrival first

FOR each order O

 INSERT O into P

END FOR

currentTime <- 0

```

WHILE P is not empty
  order <- REMOVE highest-priority order
  waitingTime <- currentTime
  currentTime <- currentTime + order.PreparationTime
  record order and waitingTime
END WHILE
OUTPUT preparation sequence and waiting times

```

5. Queue Trace - 12 Operations

#	Operation	FIFO	Priority Queue
1	ENQUEUE O1	[O1]	[O1]
2	ENQUEUE O2	[O1,O2]	[O2,O1]
3	ENQUEUE O3	[O1,O2,O3]	[O2,O1,O3]
4	ENQUEUE O4	[O1,O2,O3,O4]	[O2,O4,O1,O3]
5	ENQUEUE O5	[O1,O2,O3,O4,O5]	[O2,O4,O1,O3,O5]
6	DEQUEUE	O1 -> [O2,O3,O4,O5]	O2 -> [O4,O1,O3,O5]
7	DEQUEUE	O2 -> [O3,O4,O5]	O4 -> [O1,O3,O5]
8	DEQUEUE	O3 -> [O4,O5]	O1 -> [O3,O5]
9	DEQUEUE	O4 -> [O5]	O3 -> [O5]
10	DEQUEUE	O5 -> []	O5 -> []
11	PEEK/EMPTY	empty	empty
12	END	empty	empty

6. Correctness / Invariant

FIFO Invariant: ทุกครั้งก่อนวนรอบ Front คือ Order ที่ยังไม่เตรียมและมีเวลามาถึงเร็วที่สุดในกลุ่มที่เหลือ เมื่อ DEQUEUE จะลบ Front เท่านั้น จึงรักษาลำดับเดิมและทำให้ First Order -> First Prepared ถูกต้อง

Priority Invariant: ทุกครั้งก่อนวนรอบ Front คือ Order ที่มี Priority สูงสุดในกลุ่มที่เหลือ และถ้า Priority เท่ากันจะเป็น Order ที่มาถึงก่อน เมื่อ REMOVE จะเลือก Order ตามกฎนี้ จึงได้ลำดับการเตรียมที่ถูกต้อง

7. Time Complexity

Operation	FIFO Queue	Priority Queue
Insert	$O(1)$ amortized	$O(\log n)$
Remove	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Process n orders	$O(n)$	$O(n \log n)$

8. Space Complexity

ทั้งสองวิธีต้องเก็บ Order สูงสุด n รายการ จึงใช้พื้นที่ $O(n)$ Priority Queue มีโครงสร้าง heap เพิ่มเติม แต่ยังเป็น $O(n)$ ในเชิง asymptotic space complexity

9. Java Program

โปรแกรมใช้ ArrayDeque สำหรับ FIFO และ PriorityQueue สำหรับ Priority Queue โดยมี tie-breaker เป็น arrivalOrder เพื่อให้ผลของ Priority Queue deterministic และยุติธรรมภายใน Priority เดียวกัน

```
import java.util.ArrayDeque;
import java.util.Comparator;
import java.util.PriorityQueue;
import java.util.Queue;
```

```
public class FoodQueueSystem {
```

```
    static class Order {
```

```
        String id;
```

```
        String foodType;
```

```
        int preparationTime;
```

```
        int priority;
```

```
        int arrivalOrder;
```

```
        Order(String id, String foodType, int preparationTime, int priority, int arrivalOrder) {
```

```
            this.id = id;
```

```
            this.foodType = foodType;
```

```
            this.preparationTime = preparationTime;
```

```
            this.priority = priority;
```

```
            this.arrivalOrder = arrivalOrder;
```

```
        }
```

```
        @Override
```

```
        public String toString() {
```

```
            return id + "(" + foodType + ", " + preparationTime + " min, P" + priority + ")";
```

```
        }
```

```
    }
```

```
    static int priorityOf(String foodType) {
```

```
        if (foodType.equalsIgnoreCase("Express")) return 1;
```

```

    if (foodType.equalsIgnoreCase("Normal")) return 2;
    return 3; // Large
}

```

```

static void runFIFO(Order[] orders) {
    Queue<Order> q = new ArrayDeque<>();
    System.out.println("=== Algorithm A: FIFO Queue ===");
    for (Order o : orders) q.offer(o);
    int time = 0;
    while (!q.isEmpty()) {
        Order o = q.poll();
        System.out.println("Prepare " + o + " | start=" + time + " | wait=" + time + " min");
        time += o.preparationTime;
    }
}

```

```

static void runPriorityQueue(Order[] orders) {
    PriorityQueue<Order> pq = new PriorityQueue<>(
        Comparator.comparingInt((Order o) -> o.priority)
            .thenComparingInt(o -> o.arrivalOrder)
    );
    System.out.println("=== Algorithm B: Priority Queue ===");
    for (Order o : orders) pq.offer(o);
    int time = 0;
    while (!pq.isEmpty()) {
        Order o = pq.poll();
        System.out.println("Prepare " + o + " | start=" + time + " | wait=" + time + " min");
        time += o.preparationTime;
    }
}

```

```

public static void main(String[] args) {
    Order[] orders = {
        new Order("O1", "Normal", 15, priorityOf("Normal"), 1),
        new Order("O2", "Express", 5, priorityOf("Express"), 2),
        new Order("O3", "Normal", 10, priorityOf("Normal"), 3),
        new Order("O4", "Express", 8, priorityOf("Express"), 4),
        new Order("O5", "Large", 25, priorityOf("Large"), 5)
    }
}

```

```

};

runFIFO(orders);
System.out.println();
runPriorityQueue(orders);
}
}

```

10. Test Cases - 6 กรณี

TC	Input	Expected Result
TC01	O1 Normal	A: O1 / B: O1
TC02	O1 Normal, O2 Normal	A: O1->O2 / B: O1->O2
TC03	O1 Normal, O2 Express	A: O1->O2 / B: O2->O1
TC04	O1 Express, O2 Normal, O3 Large	B: O1->O2->O3
TC05	Scenario O1-O5	A: O1->O2->O3->O4->O5 / B: O2->O4->O1->O3->O5
TC06	O1 Express, O2 Express, O3 Normal	B: O1->O2->O3

11. Experimental Comparison

สมมติฐานของการทดลอง: Order ทั้งหมดอยู่ในระบบพร้อมกันที่เวลา 0 เพื่อเปรียบเทียบผลของกฎการจัดคิวอย่างตรงไปตรงมา และ Waiting Time = เวลาก่อนเริ่มเตรียมอาหาร

n	Algorithm	Average Wait	Maximum Wait	Fairness Gap
5	FIFO	25.0 min	55.0 min	7.5 min
5	Priority Queue	16.8 min	38.0 min	35.5 min
10	FIFO	56.5 min	118.0 min	7.5 min
10	Priority Queue	40.0 min	101.0 min	78.8 min
50	FIFO	308.5 min	622.0 min	7.5 min
50	Priority Queue	223.4 min	605.0 min	431.4 min

100	FIFO	623.5 min	1252.0 min	7.5 min
100	Priority Queue	452.9 min	1235.0 min	869.9 min

สำหรับ Scenario ที่กำหนด: FIFO Average Waiting Time = 20.6 นาที, Maximum = 38 นาที; Priority Queue Average Waiting Time = 16.8 นาที, Maximum = 38 นาที

12. สรุปเปรียบเทียบ Algorithm A และ B

หัวข้อ	FIFO Queue	Priority Queue
หลักการ	มาก่อนทำก่อน	Priority สูงทำก่อน
ลำดับ Scenario	O1 O2 O3 O4 O5	O2 O4 O1 O3 O5
Average Waiting	20.6 min	16.8 min
Maximum Waiting	38 min	38 min
Fairness	สูง	ต่ำกว่าเมื่อมีการแข่งคิว
Complexity	$O(n)$ total	$O(n \log n)$ total

คำถาม: การให้ Express Order แข่งคิวอย่างต่อเนื่องอาจสร้างปัญหาอะไร คำตอบคือ Normal และ Large อาจรอนานมากจนเกิด Starvation หรือความไม่เป็นธรรม แนวทางลดปัญหาคือกำหนด priority aging, จำกัดจำนวน Express ที่แข่งต่อเนื่อง หรือกำหนดเพดานเวลารอขั้นต่ำ/maximum waiting SLA